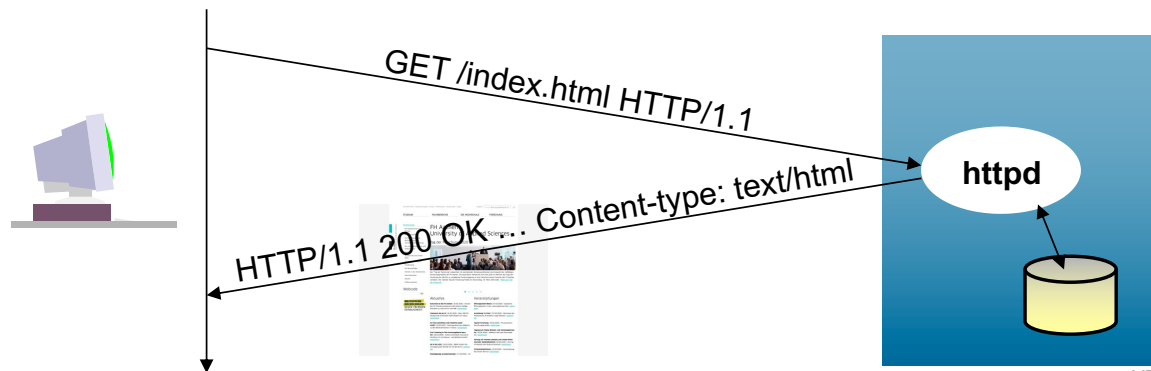


HTTP - Hypertext Transfer Protocol

Hypertext Transfer Protocol (HTTP)

Das Protokoll des WWW: HTTP

- Zustandsloses Anfrage-/Antwort-Protokoll
- Textbasiert unter Nutzung einer zeilenbasierten Struktur (pre 2.0)
- Dient der Übertragung von Ressourcen zwischen einem Server und einem Client
 - > Übertragung von Datenobjekten: Formatangaben, Inhalt, Server-Einstellungen
 - > Kodierungsregeln (analog zu E-Mail)
- Endpunkt wird über URL identifiziert (Port 80/443):
 - > Protokoll://Host:Port/Ort?Parameter



HTTP/1.1 im RFC 2616 (1999): <https://tools.ietf.org/html/rfc2616>

URI: Uniform Resource Identifier

- Eine URI dient der eindeutigen Adressierung von abstrakten und physikalischen Ressourcen im Internet (Spezifikation in RFC2396)
- URLs spezifizieren auch, wie zugegriffen wird
- URI: URLs $\cup$ URNs

URI (Uniform Resource Identifier)

URL (Uniform Resource Locator)

*Adressierung von Informationsobjekten mit Festlegung des Zugangs-
Protokolls (Ort der Ressource). RFC2141*

URN (Uniform Resource Name)

*Adressierung von Objekten ohne ein Protokoll festzulegen
(Eindeutige und gleichbleibende Referenz – Name der Ressource). RFC1738*

Weitere Informationen: <https://www.ietf.org/rfc/rfc3986.txt>

Unterschied URL/URI

- URI beschreibt allgemeinere Möglichkeit der Adressierung
- Jede URL ist auch eine URI
- Eine URL identifiziert Ressourcen eindeutig und gibt dabei gleichzeitig das **Zugriffsprotokoll** an

Beispiele:

- URI: example.com
- URI (oder eine relative URL): datei.txt
- URL: http://www.example.com/datei.txt
- URL: mailto:test@example.com
- URL: ftp://127.0.0.1/dump.sql
- URN: urn:isbn:0596517742

> URLs und URNs sind auch gleichzeitig gültige URIs

URL-Syntax

- `<Schemata>: <Schemata-spezifischer-Teil>`
- Viele URL-Schemata besitzen einen hierarchischen Aufbau.

Beispiel: HTTP URL

`http://<user>:<password>@<host>:<port>/<url-path>?<query-parameter>`

`http://user:pw@example.com:80/datei.html?q=123`

Relative URLs

- Auch die Nutzung relativer URLs wurde definiert
- Beispiel: `../datei.html`
- Nicht relative URLs werden als „absolut“ bezeichnet

Weitere Informationen: <https://www.ietf.org/rfc/rfc1738.txt>, <https://tools.ietf.org/html/rfc1808>

Basis-URL

- Für das Verständnis relativer URLs ist das Analogon „Working Directory“ passend
- Die „Basis-URL“ einer mittels einer absoluten URL-spezifizierten Internet-Ressource beinhaltet alle Zeichen bis (einschließlich) zum letzten Schrägstrich (/) im Pfadnamen

Absolute URL	Basis URL
<code>http://www.fh-aachen.de/abt_juelich.html</code>	<code>http://www.fh-aachen.de/</code>
<code>http://www.example.com/dir/inhalt.html</code>	<code>http://www.example.com/dir/</code>

Relative URL

- Eine partielle (relative) URL ist immer dann vorhanden, wenn Schemaname und ":" fehlen (**gilt nicht für die URL-Leiste im Browser!**)
- Aus der Kombination von Basis-URL (die, der aktuell angezeigten Seite im Browser) und relativer URL lässt sich immer eine absolute URL ableiten

Beispiel:

- Basis-URL: `http://www.example.com/html/`

Relative URL	Absolute URL
<code>about.html</code>	<code>http://www.example.com/html/about.html</code>
<code>path/</code>	<code>http://www.example.com/html/path/</code>
<code>path/file.html</code>	<code>http://www.example.com/html/path/file.html</code>
<code>/</code>	<code>http://www.example.com/</code>
<code>//example.org/</code>	<code>http://example.org/</code>
<code>/config/</code>	<code>http://www.example.com/config/</code>
<code>../</code>	<code>http://www.example.com/</code>
<code>../dir/</code>	<code>http://www.example.com/dir/</code>
<code>../../..</code>	<code>http://www.example.com/</code>
<code>./</code>	<code>http://www.example.com/html/</code>
<code>./about.html</code>	<code>http://www.example.com/html/about.html</code>

Ressourcennamen (URLs, Dateinamen)

- RFC3986 erlaubt die Verwendung von Nicht-ASCII-Zeichen, allerdings nur indirekt durch die sogenannte Prozentkodierung
 - > Nationale Sonderzeichen werden durch ihre hexadezimale Darstellung entsprechend ASCII-Zeichensatz dargestellt. Zur Erkennung dieser Hex-Zeichen wird ein "%" vorangestellt (URL Encoding)
 - Beispiele: ß wird zu %DF,] zu %5D, } zu %7D
 - > Leerzeichen werden zu einem „+“ kodiert („%20“ funktioniert alternativ)
- Es empfiehlt es sich **nur ASCII-Zeichen** als Dateiname zu nutzen. Andere Zeichen können zwar verwendet werden, sie werden aber vom Browser umkodiert. Beim Einsatz ohne Browser gibt es bei Copy-Paste von Dateinamen mit Leerzeichen (Windows!) Probleme
- Diverse ASCII-Zeichen wie u.a. % + & = : haben eine Sonderbedeutung und müssen ebenso mittels % kodiert werden (z.B. %26 für &)

Browser zeigen heutzutage häufig die dekodierten Werte an, was dazu führt, dass der Nutzer meist nichts von der Umwandlung merkt und mit den Umlauten arbeiten kann

Weitere Informationen: <https://tools.ietf.org/html/rfc3986>

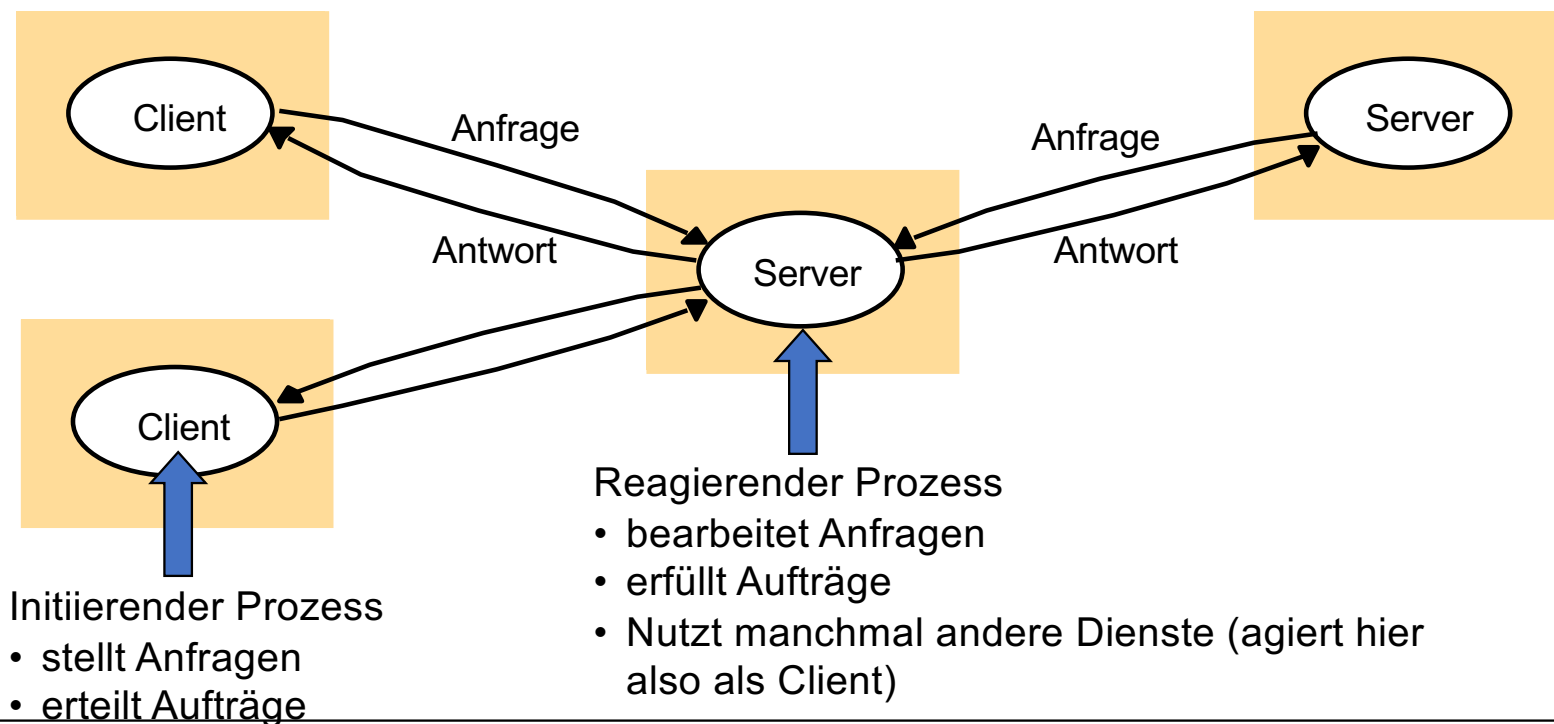
Im ISO/OSI-Modell (Siehe Kommunikationssysteme)

	Schicht	Einheit	Protokoll
7	Application	Anwendung	HTTP
6	Presentation		
5	Session		
4	Transport	Transport	TCP
3	Network	Internet	IP
2	Data Link	Netzzugriff	Ethernet
1	Physical		

HTTP

Client-Server-Modell

- Ein kurzlebiger Client-Prozess stellt Anfragen an einen langlebigen Server-Prozess
- Die Mehrzahl aller verteilten Anwendungen arbeiten nach dem Client / Server Modell



HTTP

Client-Server-Modell

- Client sendet eine *Request*
- Server antwortet mit einer *Response*
- Leider ist dies ein nicht immer ausreichendes Kommunikationsmuster

Eigenschaften

- Ein **Server ist langlebig** und stellt einen Dienst zur Verfügung
 - > Klassisch Anfrage-Antwort-Zyklus
 - > Webseite: Browser (Client) erfragt Ressource, Server antwortet (Response)
- Ein Server kann bei der Beantwortung der Anfrage zwischenzeitig als Client gegenüber einem anderen Server agieren
 - > Beispiel: Proxy-Server, Datenbank
- Begriff „Server“ wird häufig sowohl für den Dienst (z.B. Apache) als auch für die Maschine auf der der Dienst angeboten wird genutzt
- **Zustandslosigkeit vereinfacht den Server.**
 - > Die Anfrage enthält alles an Information, was man zur Bearbeitung benötigt, ggf. durch Verweise auf zwischengespeicherte Information
 - > Damit hat der Server keine unmittelbaren Abhängigkeiten über Aufrufgrenzen hinweg
 - > Vereinfacht beispielsweise einen Neustart

Ursprünglicher Ablauf

1. Verbindungsaufbau

Der Client baut eine TCP/IP-Verbindung zum Server auf (siehe Kommunikationssysteme)

2. Request

Der Client sendet eine HTTP-Anfrage (Kommando) ab. Hierbei wird die URL und die Protokoll-Version abgesetzt. Weitere Header-Elemente oder Anfrageparameter steuern das Verhalten.

3. Response

Der Server antwortet mit der Protokoll-Version und einem Statuscode enthält, der Erfolg bzw. Misserfolg der Anforderung mitteilt. Neben steuernden Header-Elementen folgt, sofern angefordert, das eigentliche Dokument in dem entsprechenden Datenformat (HTML).

4. Schließen der Verbindung

Bei HTTP 1.0 wurde die Verbindung geschlossen. Seit **HTTP/1.1** kann der Client mittels „**keep-alive**“ alternativ mitteilen, dass die **TCP/IP-Verbindung offen** bleiben soll, um zukünftige Anfragen zu beschleunigen. Die Schritte 4 und 1 entfallen dann bei zukünftigen Anfragen.

HTTP

Keep Alive bei HTTP1.1

Timeout:

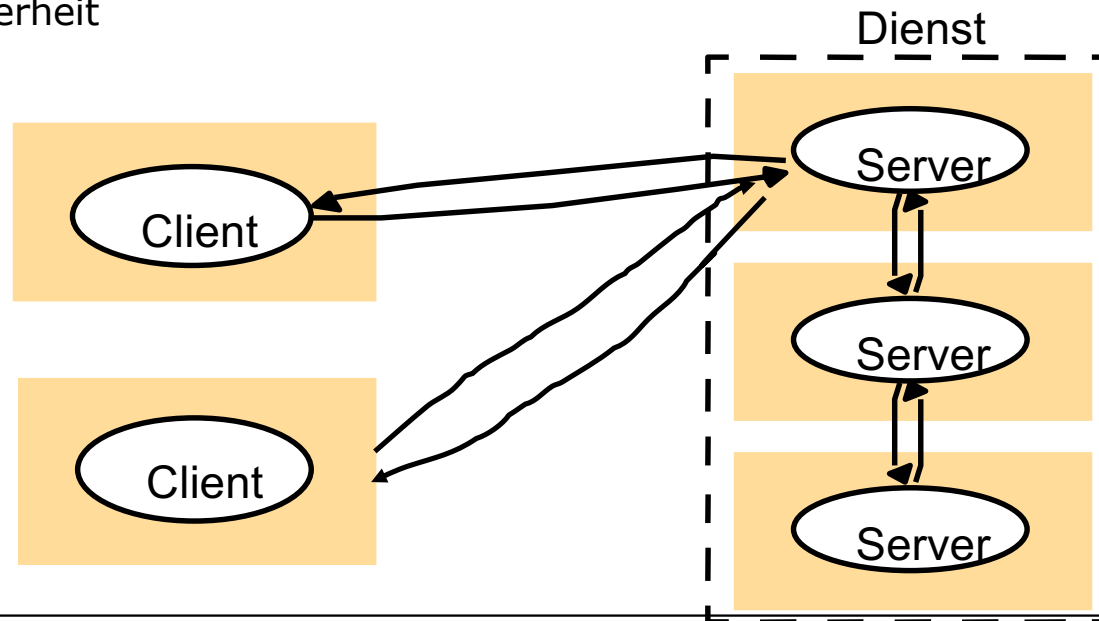
- Gibt die maximale Zeitspanne (in Sekunden) an, wie lange eine Verbindung nach der letzten Aktivität offen bleibt, bevor sie geschlossen wird. Typische Werte liegen zwischen 5 und 15 Sekunden, abhängig von der Serverkonfiguration und dem Traffic.

Maximale Anzahl von Anfragen (MaxKeepAliveRequests):

- Definiert die maximale Anzahl von Anfragen, die über eine einzige Verbindung gesendet werden können. Übliche Werte sind zwischen 50 und 100, abhängig von der Größe des Webprojekts und der Serverkapazität

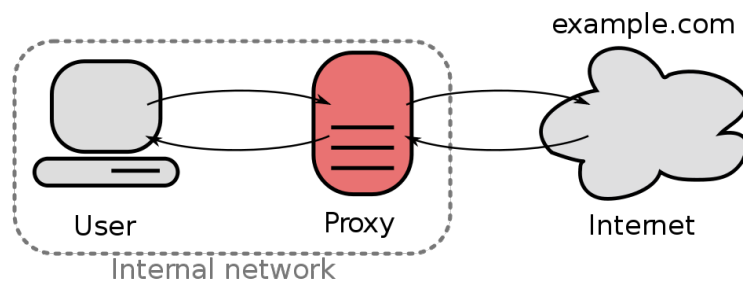
Replizierte Server

- Es werden Replikate von Serverprozesse zur Verfügung gestellt
- Jeder Server hat hier quasi die gleiche Sicht!
- Beispiele:
 - > Google, eBay, ...
 - > Transparente Replikate in Clustern zur Verbesserung der Performance und zur Ausfallsicherheit



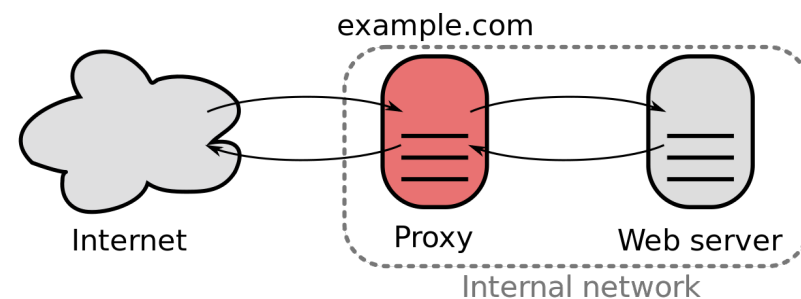
(Forward) Proxy und Reverse-Proxy Modell

- Ziele
 - > Verbesserung der Performance
 - > Filter und Kontrollpunkt
- Proxy zum Zwischenspeichern oder Lastbalanzieren von Webseiten



Forward Proxy

https://upload.wikimedia.org/wikipedia/commons/thumb/1/19/Forward_proxy_h2g2bob.svg/500px-Forward_proxy_h2g2bob.svg.png



Reverse Proxy

https://upload.wikimedia.org/wikipedia/commons/thumb/6/67/Reverse_proxy_h2g2bob.svg/1280px-Reverse_proxy_h2g2bob.svg.png

Content Delivery Networks

CDN

Viele Webseiten binden komplexe Ressourcen ein, z.B. CSS-Frameworks wie Bootstrap oder (später) JavaScript-Bibliotheken

Damit dieses Einbinden hinreichend schnell und skalierbar ist sollte eine Verteilungsstrategie realisiert werden, so dass die **Bibliotheken nicht alle vom gleichen Server geladen** werden

Ein CDN bietet einen Dienst an, um den Inhalt auf multiple Server such geografisch zu replizieren

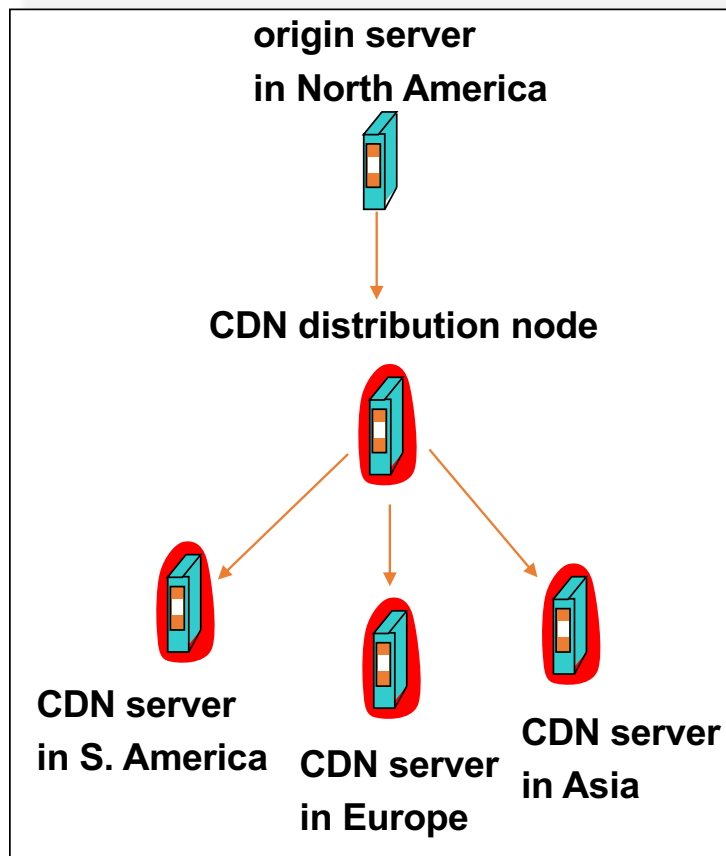
Clients nutzen (automatisch) den Server, der den Inhalt am schnellsten liefert (z.B. durch **IP-Anycast** oder **GeoDNS** – z.B. google – eher Master)

Wir erhalten eine Lastverteilung in Anlehnung an die Netztopologie

Hinweis: Faktisch gibt es mehrere Server mit dem gleichen DNS-Namen (ggf. sogar der gleichen IP-Adresse)

Content Delivery Networks

CDN



CDNs nutzen verschiedene Methoden um die angeforderten Quellen von einem “nahen” Server zu laden.

Dabei gehen die Anfragen an einen CDN Distribution Node (es gibt viele – Anycast/GeoDNS) der dann eine Lastbalancierung mittels HTTP-Redirects vornimmt.

Aus Kurose-Ross: Computer Networking: A Top-Down Approach

Content Delivery Networks

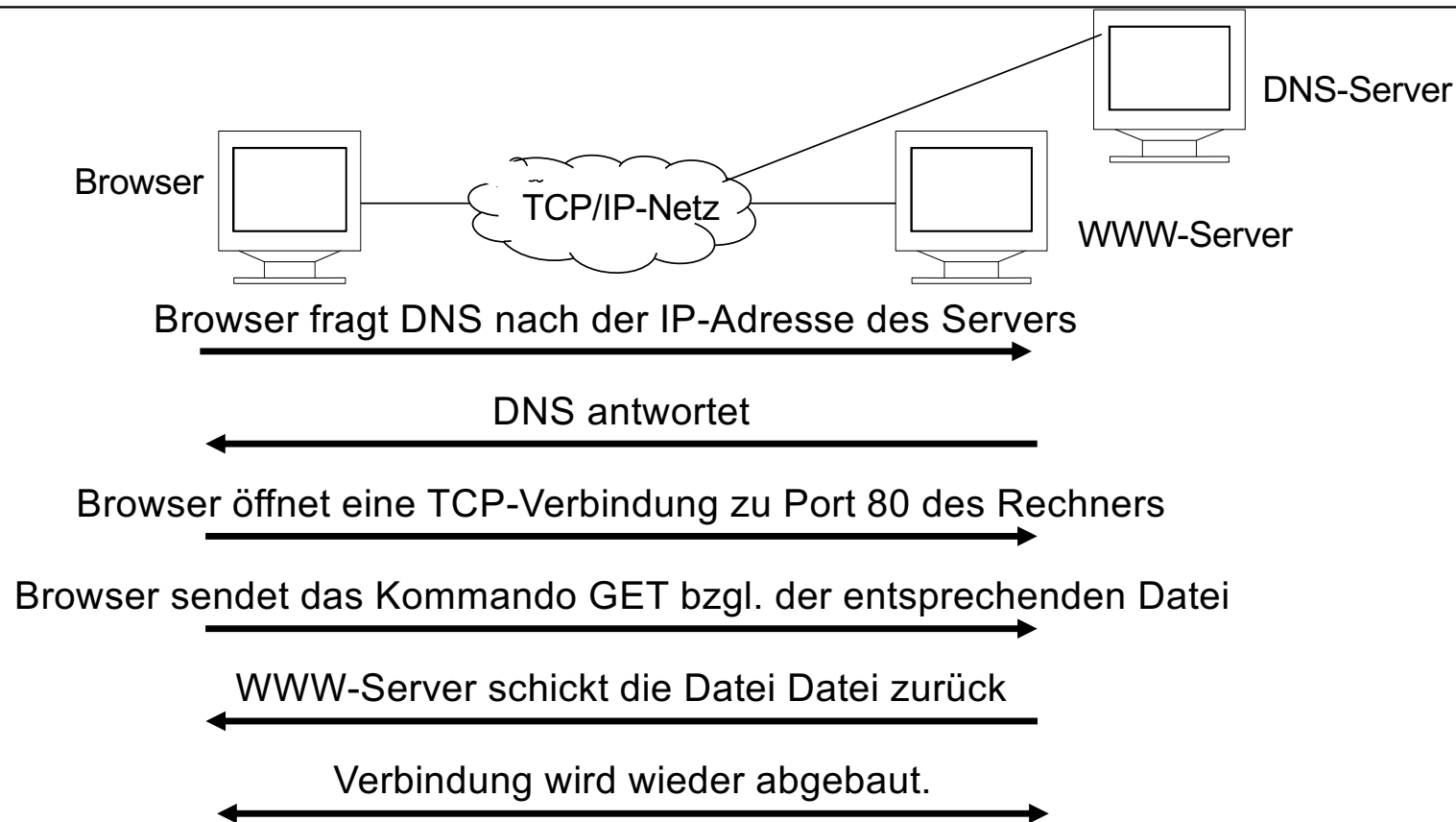
Ausblick

Auch wenn Sie nach der Vorlesung Kommunikationssysteme erkennen, dass eigentlich UDP das geeignete Protokoll zur Übertragung von Streaming-Daten ist, so ist die existierende CDN-Infrastruktur und Technologie so bedeutend, dass die bekannten Streamingplattformen http-basiert diese Technik nutzen

Näheres im Master

HTTP

Ablauf einer 1.0 Anfrage



HTTP

Request und Response

Formaler Aufbau von Request und Response

Request = Request-Line
 *((general-header | request-header | entity-header)) <CRLF>
 [message-body]

Response = Status-Line
 *((general-header | response-header | entity-header)) <CRLF>
 [message-body]

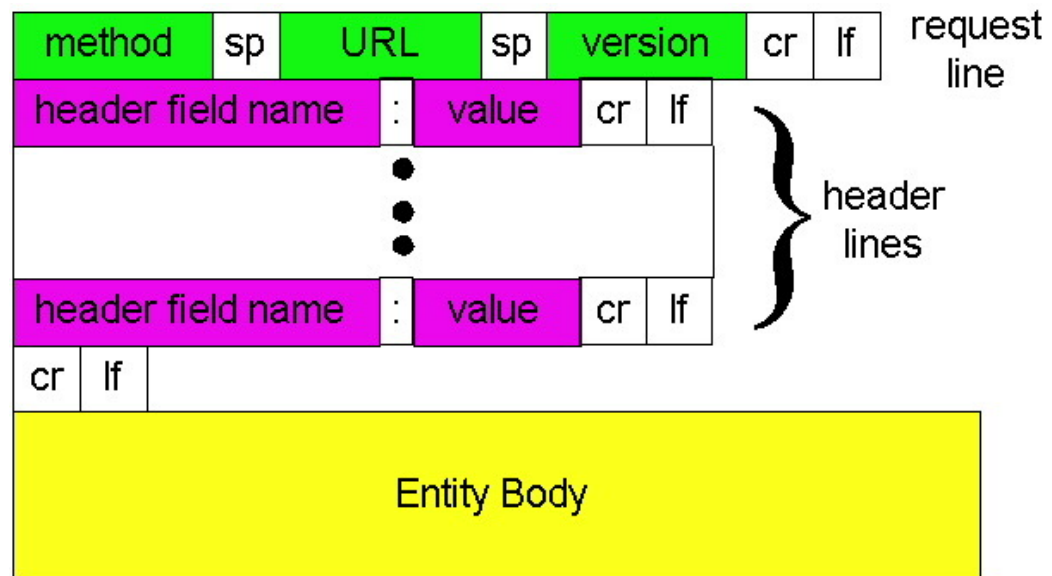
Request-Line = Method *blank* Request-URI *blank* HTTP-Version <CRLF>

Status-Line = HTTP-Version *blank* Status-Code *blank* Reason-Phrase <CRLF>

HTTP/1.1 im RFC 2616 (1999): <https://tools.ietf.org/html/rfc2616>

HTTP

Request



Quelle: <http://userpages.umbc.edu/~dgorin1/451/OSI7/dcomm/http.htm>

Request-Beispiel (GET):

GET <https://www.fh-aachen.de/> HTTP/1.1

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9

Accept-Encoding: gzip, deflate, br

Accept-Language: de-DE,de;q=0.9,en-US;q=0.8,en;q=0.7

Cache-Control: no-cache

Connection: keep-alive

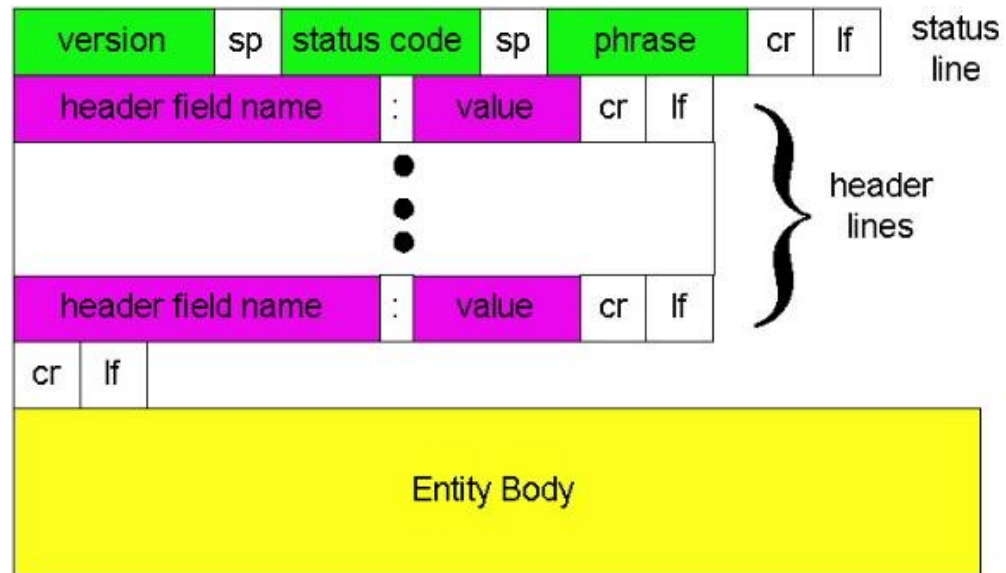
Cookie: _pk_ref.4.19d4=%5B%22%22%2C%22%22%2C1670832293%2C%22https%3A%2F%2Fwww.google.com%2F%22%5D;
_pk_id.4.19d4=4115fa9a7f694583.1670832293.; fhac_cookiemodal-selection=[%22essential%22%2C%22analytics%22%2C%22search%22];
fe_typo_user=bf03b5cab5fc0cdae56a04e56a4413a5

Host: www.fh-aachen.de

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Linux; Android 6.0; Nexus 5 Build/MRA58N) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/108.0.0.0 Mobile Safari/537.36

HTTP Response



Quelle: <http://userpages.umbc.edu/~dgorin1/451/OSI7/dcomm/http.htm>

Response-Beispiel:

HTTP/1.1 200 OK

Connection: keep-alive

Content-Encoding: gzip

Content-Language: de

Content-Length: 13083

Content-Type: text/html; charset=utf-8

Date: Mon, 06 Feb 2023 09:17:41 GMT

Server: nginx/1.18.0 (Ubuntu)

Strict-Transport-Security: max-age=31536000

```
<!DOCTYPE html>
```

```
<html>
```

```
...
```


Statuscodes

Code	Bedeutung
1xx	Informational – Anforderung empfangen, Aktion folgt
2xx	Success - Erfolgreiche Abarbeitung der Anfrage
3xx	Redirection - Angefragtes Objekt befindet sich an anderer Stelle
4xx	Client Error - Falsche / Unerlaubte Anfrage
5xx	Server Error - Server kann Anfrage nicht ausführen

■ Beispiele:

- > 200 OK
- > 201 Created
- > 301 Moved Permanently
- > 302 Moved Temporarily
- > 404 Not Found
- > 500 Internal Server Error

Details im HTTP/1.1 RFC: <https://www.w3.org/Protocols/rfc2616/rfc2616-sec10.html>

HTTP

Request-Methoden

Methode	Bedeutung
GET	Anforderung, der im Request-URI angeforderten Ressource
POST	Wird zur Übertragung von Daten an den Server benutzt
HEAD	Anforderung des Headers einer Ressource (wie GET, lässt aber den Seiteninhalt weg)
PUT	Wird zur Übertragung einer neuen Ressource an den Server benutzt.
DELETE	Löschen von Dokumenten auf dem Server. Der Server muss entsprechende Rechte auf dem Server besitzen (Sicherheitsprobleme)
OPTIONS	Diese Methode erlaubt, den Zugriffspfad zu einer Ressource zu ermitteln und die Methoden, die darüber möglich sind.
TRACE	Für Testzwecke: Die Anfrage wird vom Server zurück geschickt

- Zunächst werden nur GET und POST genutzt, teilweise HEAD
- Andere Methoden wurden insbesondere durch REST-Anwendungen populär (neue patch-Methode)

Details im HTTP/1.1 RFC: <https://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html>

Eigenschaften

- Übertragung der Daten über die Adresszeile (**kein HTTP-Body**)!
- Parameter werden durch das Zeichen ? in der URL eingeleitet
- Aufruf kann als Lesezeichen gespeichert werden
- **Nicht geeignet zur Übertragung großer Datenmengen**
- Daten, die per GET übertragen werden, werden häufig geloggt (z.B. vom Server oder einem Proxy) und möglicherweise auch vom Browser gespeichert (im Cache)
 - > **Keine sensiblen Daten per GET übermitteln**
 - > Sichtbar in der URL

Beispiel

```
GET /index.php?page=welcome&id=42 HTTP/1.1
Host: www.example.com
...
Connection: keep-alive
```

HTTP

POST-Request

Eigenschaften

- Übertragung der Daten im HTTP-Body (≠ HTML-Body!)
 - > Daten sind nicht in der URL sichtbar
 - > Content-Length-Header zur Angabe der Länge erforderlich
- Geeignet für große Datenmengen, freies Encodieren
- Zusätzlich möglich: Übermittlung von Daten in der URL analog zur GET-Methode
- Daten, die per POST übertragen werden, werden i.d.R. nicht mitgeloggt und auch nicht in einem Cache vorgehalten
 - > Zusätzliche Verschlüsselung ist trotzdem sinnvoll
- **Leerzeile** trennt HTTP-Header von body (wie bei der Response)

Beispiel

```
POST /edit.php HTTP/1.1
```

```
Host: www.example.com
```

```
...
```

```
Connection: keep-alive
```

```
username=vosa&pw=supersecret123
```

Beispiel POST

```
<form action="do.php?q=login" method="post">
  <input type="text" name="username" />
  <input type="password" name="pw" />
  <input type="submit" value="Login" />
</form>
```

- Erzeugter HTTP-Request (bei entsprechender Eingabe):

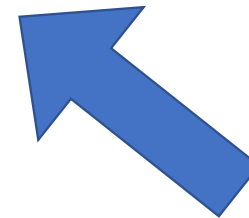
```
POST /do.php?q=login HTTP/1.1
Host: www.example.com
...
Content-Type: application/x-www-form-urlencoded
Content-Length: 31
Connection: keep-alive

username=vosa&pw=supersecret123
```

Weitere Informationen: <https://developer.mozilla.org/docs/Web/HTML/Element/form>

Beispiel GET

```
<form action="do.php?q=login" method="get">  
  <input type="text" name="username" />  
  <input type="password" name="pw" />  
  <input type="submit" value="Login" />  
</form>
```



▪ Erzeugter HTTP-Request:

```
GET /do.php?username=thomas&pw=supersecret123 HTTP/1.1  
Host: www.example.com  
...  
Connection: keep-alive
```

Weitere Informationen: <https://developer.mozilla.org/docs/Web/HTML/Element/form>

HTML Elemente

Formulare

```
<form action="do.php?q=login" method="post">
  <input type="text" name="username" />
  <input type="password" name="pw" />
  <input type="submit" value="Login" />
</form>
```

- Problem: Wozu würde oben `method="get"` führen?
 - > `do.php?username=...&pw=...`
 - > `q=login` würde wegfallen, da URL überschrieben wird

HTML Elemente

Formulare (ohne ausführliche Erklärung)

- Auswahl an Eingabemöglichkeiten via `<input>`

<code><input type="..."></code>	Bedeutung
<u>text</u>	Einfaches (einzeiliges) Eingabefeld
<u>password</u>	Eingabefeld, das die Eingabe verschleiert [*****]
<u>submit</u>	Button zum Übermitteln des Formulars, gute Alternative: <u><button></u>
<u>radio</u>	„Schaltknöpfe“ von denen genau ein Feld ausgewählt werden kann
<u>checkbox</u>	Mehrfachauswahl über Kästchen
<u>file</u>	Auswahl einer Datei zum Upload
<u>hidden</u>	Verstecktes Feld, das dem Nutzer nicht angezeigt wird
<u>number</u>	Eingabefeld für eine Zahl
<u>date</u>	Eingabefeld für Datum
<u>datetime</u>	Eingabefeld für Datum und Uhrzeit
<u>email</u>	Eingabefeld für eine Emailadresse
<u>range</u>	Slider zur Auswahl in einem festgelegten Bereich
<u>url</u>	Eingabefeld für eine URL

Vollständige Liste mit weiteren Live-Beispielen: <https://wiki.selfhtml.org/wiki/HTML/Formulare>

HTTP 1.0

Ursprüngliches HTTP/1.0 Protokoll von 1996

- RFC 1945: <https://tools.ietf.org/html/rfc1945>
- Pragmatisches, einfaches Protokoll auf Basis des zuverlässigen verbindungsorientierten Transportprotokolls TCP
- ASCII/Text-basiert
- Jede einzelne Operation wird über eine separate TCP-Verbindung realisiert
 - > TCP CONNECT => GET index.html => TCP CLOSE
 - > TCP CONNECT => GET image.html => TCP CLOSE
 - > ...
 - > Je eingebundener Ressource ist ein weiterer TCP-Handshake notwendig
 - > Großen **Aufwand** beim sukzessiven Laden mehrerer Ressourcen, insbesondere auch **durch den** damit verbundenen **TCP-SlowStart** (siehe Kommunikationssysteme)

HTTP 1.1

HTTP/1.1 wurde 1999 standardisiert

- Ursprünglicher RFC 2616: <https://tools.ietf.org/html/rfc2616>
 - > Formulierungen in RFCs 7230-7235 verbessert

- **Persistente Verbindungen**
 - > Der Client kann nun mittels „Connection: keep-alive“ angeben, dass er gerne die TCP/IP-Verbindung offen halten möchte
 - > Kompliziertere Implementierung durch Zustand, aber deutliche Verbesserung beim Laden

- Virtual Hosts
 - > Auf einem Rechner sollen verschiedene Domains und Dienste zur Verfügung stehen
 - > Vorherige Lösungen (HTTP/0.9) sahen verschiedene Server-Ports pro Dienst oder auch mehrere IP-Adressen für einen Rechner vor
 - > **Non-IP-based Virtual Hosts** lösten dieses Problem endgültig
 - Im HTTP Request muss der Header *Host* enthalten sein
 - Beliebig viele Domains können sich dann die gleiche IP-Adresse teilen

HTTP

Binärdaten und das textbasiert: MIME

HTTP ist textbasiert (eMail auch!)

Problem: Wie können binäre Daten übertragen werden?

- Die Antworten des Servers auf eine vollständige GET-Request beinhaltet MIME-Informationen
- MIME = Multipurpose Internet Mail Extensions
- Definiert die Kodierungsregeln für Nicht-ASCII-Nachrichten
- Andernfalls werden NullBytes beispielsweise als Stringende interpretiert (Programmiersprache C)
- MIME ermöglicht die Nutzung verschiedener Kodierungen (media types) in einer Nachricht

Die “Content-Type:”-Zeile legt den Datentyp (type/subtype) fest

Content-Transfer-Encoding: definiert im MIME-Standard die Transfersyntax, wird aber bei HTTP nicht benutzt

Stattdessen wird der **Content-Encoding: Header** und der **Transfer-Encoding: Header** verwendet

Beispiele:

- Content-Type: text/html
- Content-Type: image/GIF
- Content-Encoding: identity

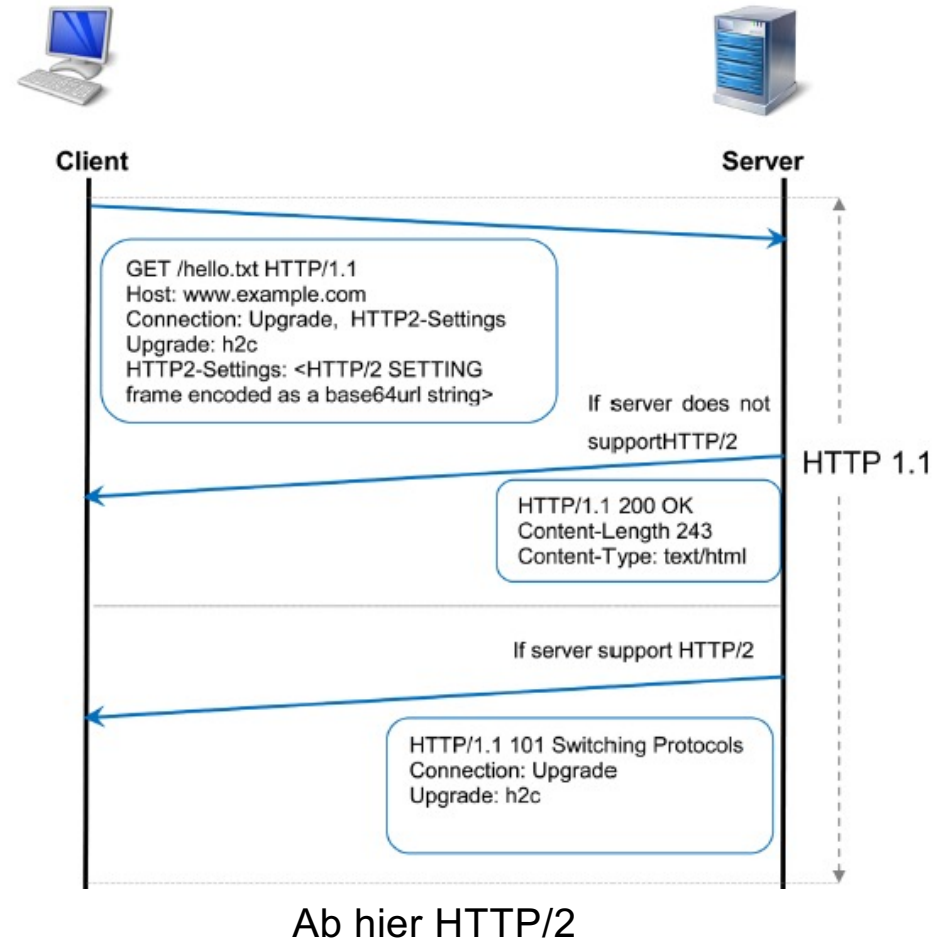
HTTP 2.0

HTTP/2 wurde 2015 veröffentlicht

- RFC 7540 (und 7541): <https://tools.ietf.org/html/rfc7540>
- Beschleunigung der Datenübertragung
- Aber: Abwärtskompatibel zu HTTP/1.1 (das übliche URL-Schema bleibt erhalten) -> Upgrade der Verbindung
- Trotzdem ist die Übertragung der Protokollparameter binär
- Wichtige Neuerungen
 - > **Multiplexing**: Mehrere Anfragen können, ohne auf eine Antwort warten zu müssen, gestellt werden. Grundidee hierbei ist, dass **der Anfrage-Stream für weitere Anfragen offen bleiben kann**, bis ein END_STREAM-Flag verwendet das Ende anzeigt. Analog für die Antworten. Damit werden immer Persistente Verbindungen genutzt! Der **Keep-Alive-Header entfällt** wieder
 - > **Header-Compression**, um die zunehmend langen geschwätzigen Header effektiver zu übertragen
 - > **Server-PUSH**, also Datenübertragungen, die vom Server initiiert werden können; Einsparen weiterer Ladevorgänge

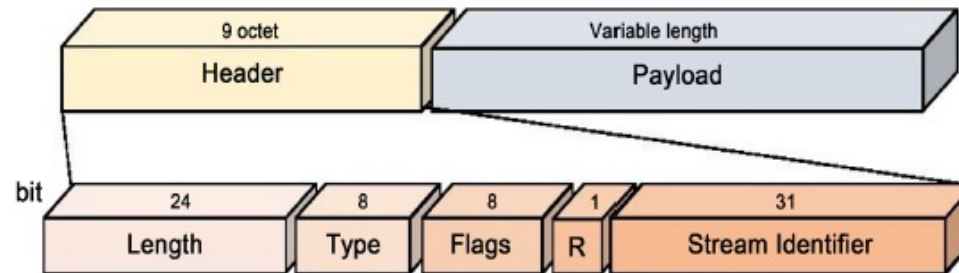
HTTP 2.0: Das Upgrade

Biljana Dimitrova, Aleksandra Mileva Journal of Computer and Communications, 2017, 5, 98-111

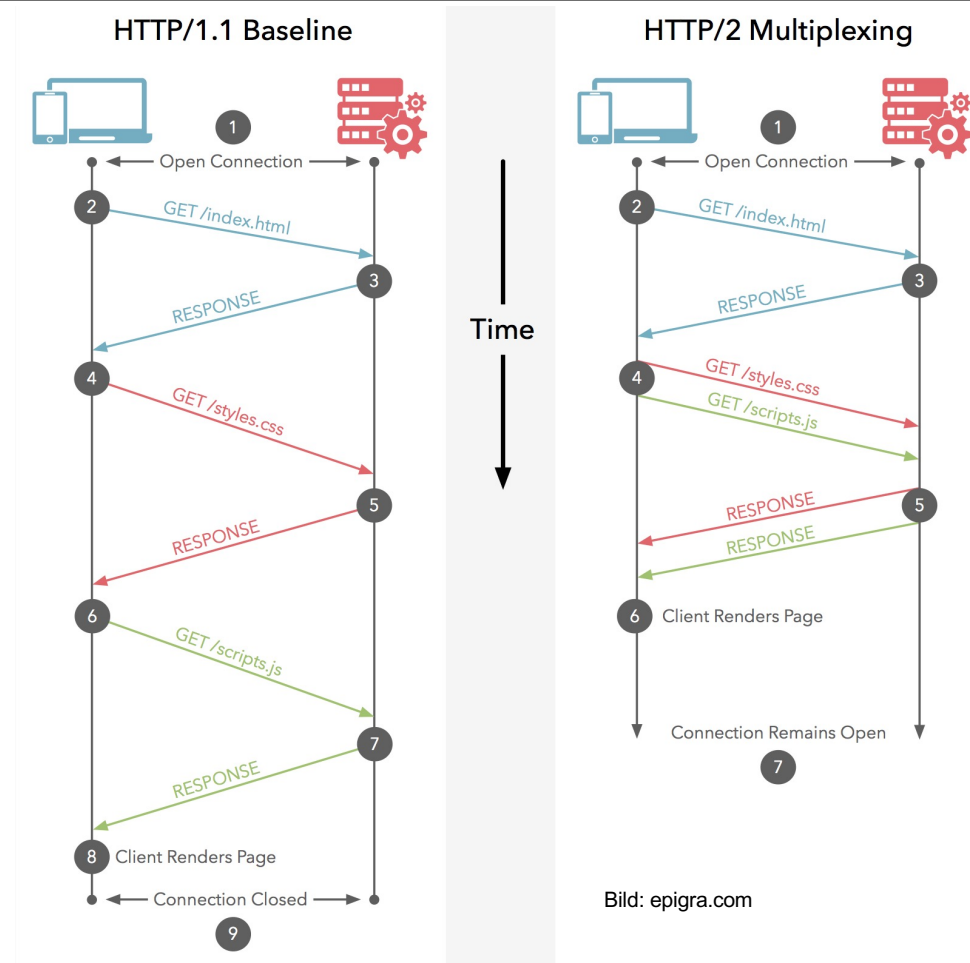


HTTP 2.0 - Multiplexing

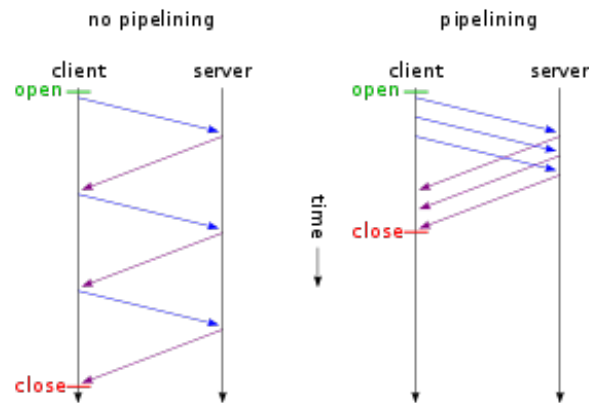
Biljana Dimitrova, Aleksandra Mileva Journal of Computer and Communications, 2017, 5, 98-111



HTTP 2.0



HTTP1.1 erlaubt Pipelining, aber...



- Implementation in web browsers
- Of all the major browsers, only Opera based on Presto layout engine had a fully working implementation that was enabled by default. In all other browsers HTTP pipelining is disabled or not implemented.[5]
- Internet Explorer 8 does not pipeline requests, due to concerns regarding buggy proxies and head-of-line blocking.[9]
- Internet Explorer 11 does not support pipelining. [10]
- **Mozilla** browsers (such as Mozilla Firefox, SeaMonkey and Camino) support pipelining; however, it is **disabled by default**. [11][12] **Pipelining is disabled by default to avoid issues with misbehaving servers**. [13] When pipelining is enabled, Mozilla browsers use some heuristics, especially to turn pipelining off for older IIS servers. [14] Support for H1 Pipeline was removed from Mozilla Firefox in Version 54. [15]
- Konqueror 2.0 supports pipelining, but it's disabled by default. [citation needed]
- Google Chrome previously supported pipelining, but it has been disabled due to bugs and problems with poorly behaving servers. [16]
- Pale Moon (web browser) supports pipelining, and is enabled by default [17]

HTTP

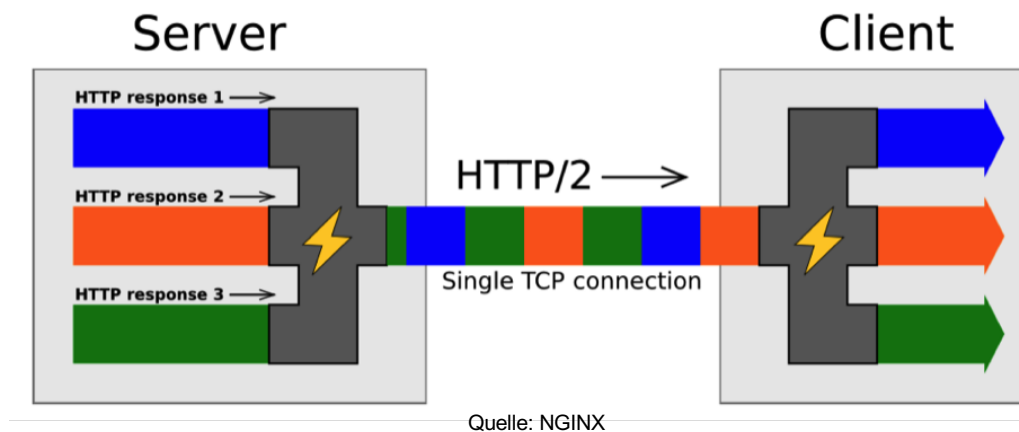
Pipelining vs. Multiplexing

https://en.m.wikipedia.org/wiki/HTTP_pipelining

Der Unterschied

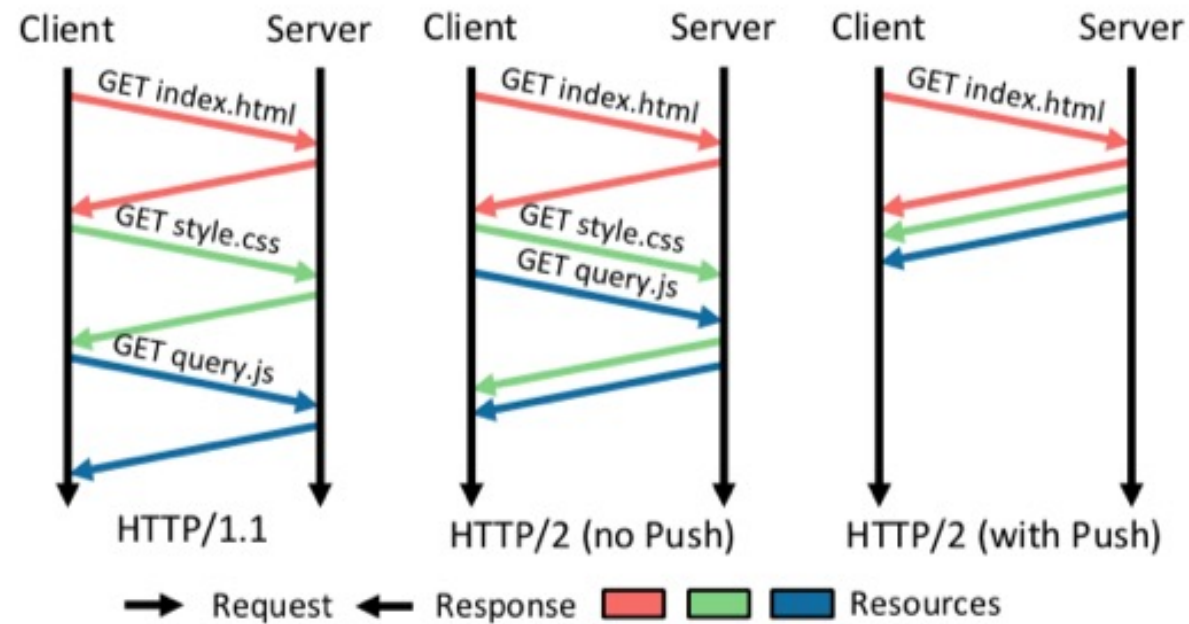
- Beim Pipelining können die Folgeanfragen gestartet werden, bevor eine Antwort erhalten wurde
- Die Antworten kommen aber in der gleichen Reihenfolge, wie die Anfragen gestartet wurden
- Beim Multiplexing können jedoch die Antworten in beliebiger Reihenfolge kommen. Letztlich werden Streams in Chunks aufgeteilt und via Multiplexing verteilt
- Insgesamt ergibt sich so eine Beschleunigung der Übertragung

HTTP/2 Inside: multiplexing



HTTP 2.0 - Server-Push

<https://blog.apnic.net/2018/04/26/adoption-performance-and-human-perception-of-http-2-server-push/>



HTTP 3.0

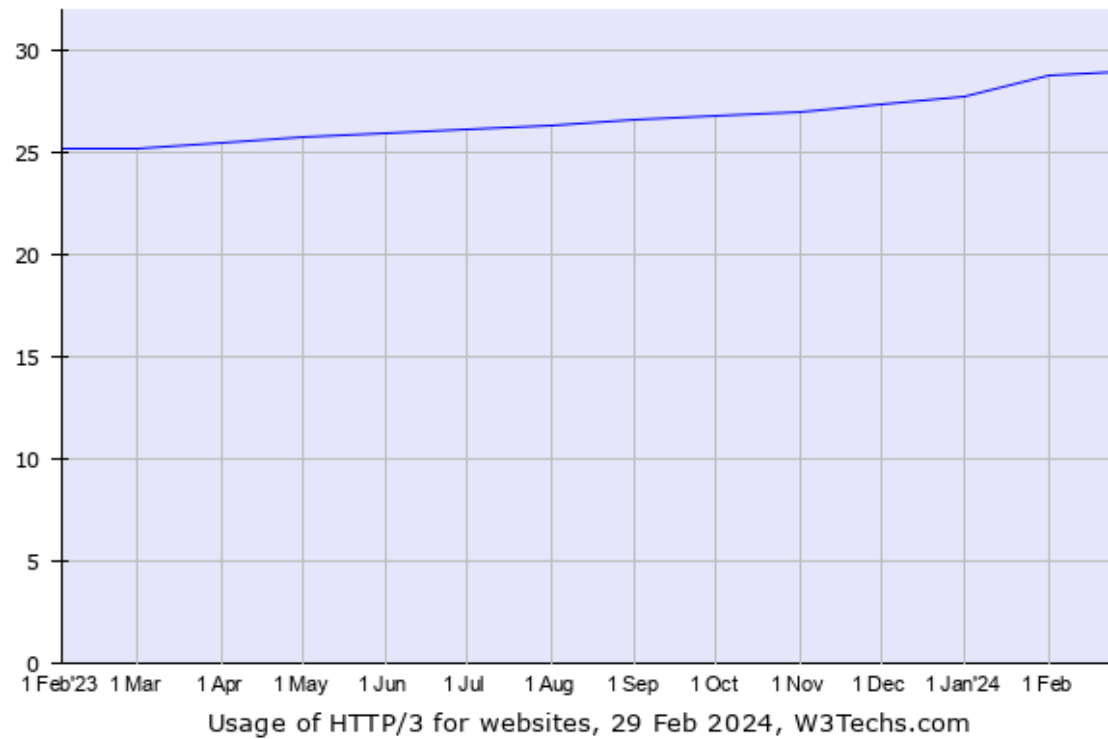
- **Seit Juni 2022 standardisiert (RFC9114)**
- Hauptänderung: Umstieg von TCP auf QUIC

QUIC:

- **basiert auf UDP** -> keine Kompatibilität zu HTTP1/2, kein Upgrade einer Verbindung
- beseitigt ineffiziente Elemente von TCP (es bei HTTP2 weiterhin ein Head-of-Line-Blocking)
- übernimmt und verbessert das Multiplexing aus HTTP/2
- integriert beim Verbindungsaufbau effizient die TLS-Handshakekonzepte
- beinhaltet eigene Fehlerkorrektur, Stau- und Flusskontrolle
- erhöht die Anonymität, da die UDP-Port-Nummern nicht mehr zwingend die Anwendung adressieren

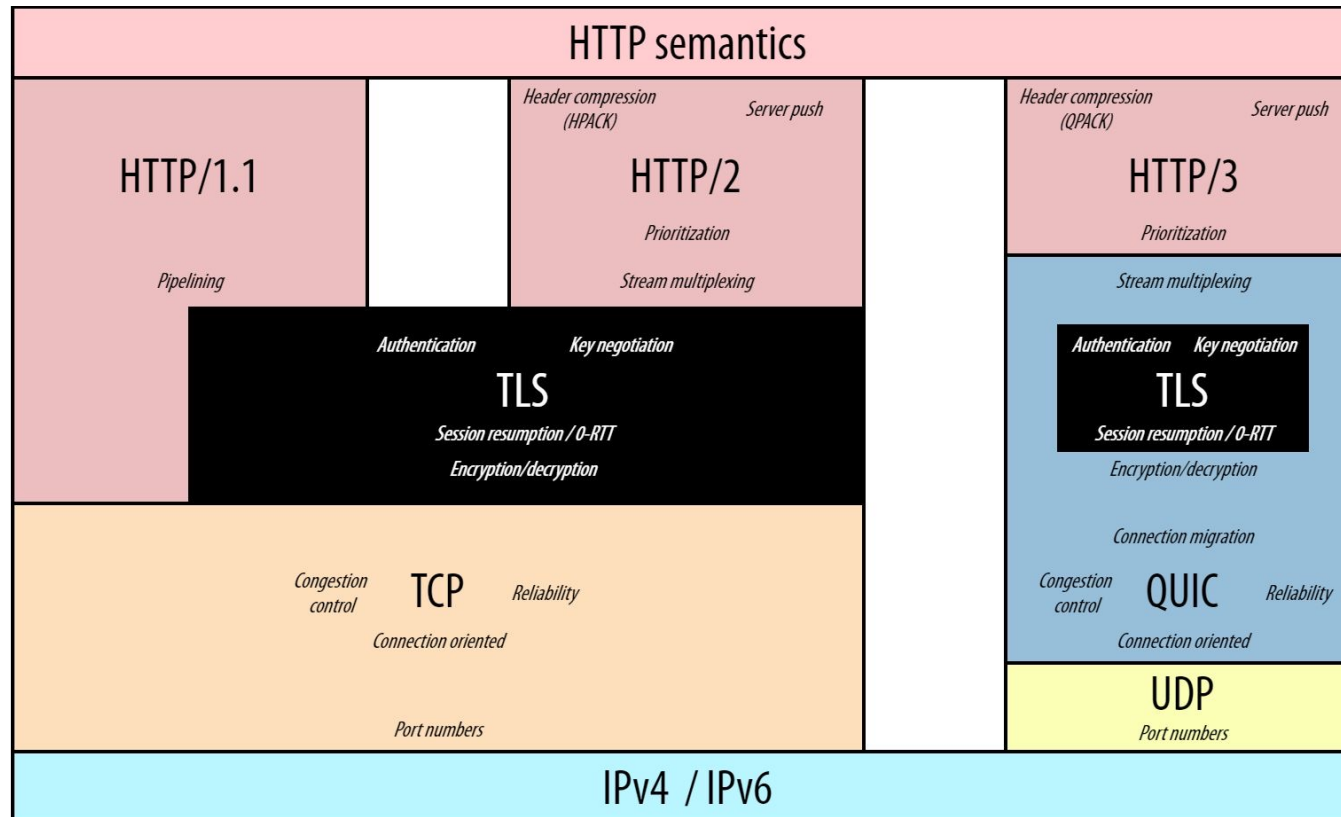
Ein Browser merkt sich die HTTP3-fähigen Server

HTTP 3.0



<https://w3techs.com/technologies/details/ce-http3>

HTTP 3.0



<https://github.com/rmarx/h3-protocol-stack/blob/main/README.md>